Cloud DevOps Engineer Learning Roadmap for 2027

Foundations | Delivery | Production operations | AI extensions

This is a practical learning sequence for building and operating cloud applications. Choose one cloud and one tool from each category before expanding. The objective is to demonstrate a working system and explain its failure modes, rather than memorize a large list of products. It is a suggested roadmap, not a forecast of job requirements or a guarantee of employment.

Use the roadmap in four stages

- Foundations: Linux, Git, networking, protocols and Python.
- Delivery: one cloud, containers, CI/CD, infrastructure as code, Kubernetes and Helm.
- Production: observability, security, reliability, recovery and cost control.
- Advanced extension: deploy and operate an AI application after the foundations are useful.

Choose your first stack

For example, choose Azure with Azure Pipelines and Terraform or OpenTofu, or AWS with GitHub Actions and Terraform. GCP is another valid starting cloud. Tool alternatives are not additional prerequisites. Add a second cloud by mapping identity, networking, compute, data and monitoring concepts after you can run a complete project in the first one.

Track evidence of progress

For each topic, keep a repository change, a short explanation and a reproducible test. Record a failure you investigated and how you verified the fix. Avoid publishing secrets, access tokens or real customer data in a portfolio.

LINUX

Foundation. Operate and troubleshoot a server.

- Files, permissions, users and groups
- Processes, signals, systemd and logs
- Packages, disks, mounts and filesystems
- CPU, memory, disk and network checks
- SSH, Bash, pipes, grep and cron
- Lab: diagnose a failed systemd service

Use Linux daily. Learn to explain the commands you run.

Completion check: demonstrate one relevant task without copying commands blindly, then explain the expected result, one failure case and how you would investigate it.

GIT COLLABORATION

Foundation. Version every change you can.

- Commits, branches and pull requests
- Merge, rebase and conflict resolution
- Tags, releases, revert and cherry-pick
- Branch protection and code review
- YAML, JSON and Markdown basics
- Lab: resolve a conflict and revert a release

A working Git workflow comes before a complex pipeline.

Completion check: demonstrate one relevant task without copying commands blindly, then explain the expected result, one failure case and how you would investigate it.

NETWORKING

Foundation. Understand how systems connect.

- IPv4, IPv6 and private/public addresses
- CIDR, subnet masks and subnet sizing
- Routes, gateways, NAT and peering
- DNS records, resolution and private DNS
- Ports, sockets and connection tracking
- Lab: trace a connection across two subnets

Calculate subnets and explain where a packet goes next.

Completion check: demonstrate one relevant task without copying commands blindly, then explain the expected result, one failure case and how you would investigate it.

PROTOCOLS

Foundation. Understand what travels over the network.

- TCP, UDP and ICMP
- HTTP/HTTPS, methods and status codes
- TLS, certificates and trust chains
- DNS, DHCP and SSH
- WebSockets and gRPC basics
- Lab: debug with curl, dig and openssl

A timeout, TLS failure and HTTP 503 need different checks.

Completion check: demonstrate one relevant task without copying commands blindly, then explain the expected result, one failure case and how you would investigate it.

FIREWALLS ACCESS

Foundation. Decide which connections are allowed.

- Stateful vs stateless filtering
- AWS security groups and network ACLs
- Azure NSGs and Azure Firewall
- GCP VPC firewall rules
- Linux nftables and Kubernetes NetworkPolicy
- Lab: allow only the required source and port

Know the source, destination, port, direction and rule scope.

Completion check: demonstrate one relevant task without copying commands blindly, then explain the expected result, one failure case and how you would investigate it.

TRAFFIC ROUTING

Foundation. Connect OSI theory to real requests.

- OSI and TCP/IP models
- L3 routing vs L4 and L7 load balancing
- Reverse proxies, ingress and API gateways
- TLS termination and health probes
- DNS routing, global routing and CDN caching
- Lab: trace DNS to load balancer to app

Use the layers to isolate failures, not just recite seven names.

Completion check: demonstrate one relevant task without copying commands blindly, then explain the expected result, one failure case and how you would investigate it.

PYTHON AUTOMATION

Foundation. Turn repeatable work into reliable scripts.

- Functions, data structures and modules
- Files, JSON/YAML and environment variables
- Exceptions, logging and unit tests
- HTTP APIs, authentication and pagination
- Cloud SDKs, retries and timeouts
- FastAPI basics and virtual environments

Project: build a resource inventory tool with useful error handling.

Completion check: demonstrate one relevant task without copying commands blindly, then explain the expected result, one failure case and how you would investigate it.

CLOUD PICK ONE FIRST

Build Next. Build depth before adding a second cloud.

- Core: IAM, networking, compute and storage
- AWS: EC2, VPC, S3, IAM, RDS and EKS/ECS
- Azure: VNet, VMs, Blob, Entra, AKS and ACA
- GCP: VPC, Compute Engine, Storage, IAM and GKE
- Databases, queues, private access and backups
- Budgets, tagging, quotas and shared responsibility

Start with AWS, Azure or GCP. Map concepts to another later.

Completion check: demonstrate one relevant task without copying commands blindly, then explain the expected result, one failure case and how you would investigate it.

DOCKER CONTAINERS

Build Next. Understand the runtime, not only the build.

- Images, layers, registries and image tags
- Dockerfiles, multi-stage builds and caching
- Container networking and persistent volumes
- Entrypoints, signals and graceful shutdown
- Non-root images and vulnerability scanning
- Compose, resource limits and debugging

Lab: build a small image and troubleshoot an unhealthy container.

Completion check: demonstrate one relevant task without copying commands blindly, then explain the expected result, one failure case and how you would investigate it.

CI CD GITOPS

Build Next. Build one complete delivery path first.

- AWS: CodePipeline and CodeBuild
- Azure: Azure Pipelines and reusable templates
- Open source: Jenkins or GitLab CI/CD
- GitHub Actions: hosted or self-hosted runners
- Build, test, scan, publish and promote
- Approvals, rollback, OIDC and Argo CD

Tools are alternatives. Learn artifacts, gates and safe delivery first.

Completion check: demonstrate one relevant task without copying commands blindly, then explain the expected result, one failure case and how you would investigate it.

INFRASTRUCTURE AS CODE

Build Next. Make infrastructure reviewable and repeatable.

- Terraform or OpenTofu: providers and resources
- Variables, outputs, modules and environments
- Remote state, locking and secret exposure
- Plan review, drift, import and lifecycle
- Azure-native Bicep; AWS CloudFormation
- Ansible for configuration management

Lab: provision, change and recover an environment through code.

Completion check: demonstrate one relevant task without copying commands blindly, then explain the expected result, one failure case and how you would investigate it.

KUBERNETES

Build Next. Learn the objects and the failure modes.

- Pods, Deployments, Services and namespaces
- ConfigMaps, Secrets, probes and rollouts
- Requests, limits, HPA and VPA
- Ingress/Gateway API, DNS and NetworkPolicy
- RBAC, ServiceAccounts, PVCs and scheduling
- Debug Pending, OOMKilled and restart loops

Lab: deploy, expose, break and recover an application.

Completion check: demonstrate one relevant task without copying commands blindly, then explain the expected result, one failure case and how you would investigate it.

HELM

Build Next. Package Kubernetes configuration cleanly.

- Charts, templates and values.yaml
- Release lifecycle and revision history
- Helpers, conditionals, loops and functions
- Chart dependencies and environment values
- helm lint, template, upgrade and rollback
- Secret handling and compatible upgrades

Helm rolls back release configuration, not business data.

Completion check: demonstrate one relevant task without copying commands blindly, then explain the expected result, one failure case and how you would investigate it.

OBSERVABILITY

Production Skill. Find the cause before changing the system.

- Metrics, logs, traces and correlation IDs
- Prometheus, Grafana and OpenTelemetry
- CloudWatch, Azure Monitor or Cloud Monitoring
- Latency, errors, traffic and saturation
- Useful alerts, dashboards and runbooks
- Lab: trace a slow request across services

Know what healthy looks like and alert on user impact.

Completion check: demonstrate one relevant task without copying commands blindly, then explain the expected result, one failure case and how you would investigate it.

SECURITY RELIABILITY

Production Skill. Operate the application after deployment.

- IAM/RBAC, least privilege and workload identity
- Secrets management, rotation and supply chain
- PostgreSQL, Redis, queues and connection pools
- SLOs, incidents, capacity and cost control
- Backups, restore tests, RPO/RTO and regional DR
- Compatible releases and safe database migrations

A backup is useful only if you can restore and validate it.

Completion check: demonstrate one relevant task without copying commands blindly, then explain the expected result, one failure case and how you would investigate it.

WINDOWS POWERSHELL

Role-Dependent. Add depth when your workloads need it.

- PowerShell scripts and remoting
- Services, Event Viewer and Task Scheduler
- NTFS permissions and service identities
- IIS, certificates and Windows networking
- Windows containers and host compatibility
- Lab: deploy and diagnose a .NET service

Learn Linux first for this path; expand Windows skills as needed.

Completion check: demonstrate one relevant task without copying commands blindly, then explain the expected result, one failure case and how you would investigate it.

AI AI NATIVE CONCEPTS

Advanced Track. Learn to deploy and operate AI applications.

- LLMs, tokens, context and structured outputs
- Embeddings, chunking, vector search and RAG
- Agents, tool calls, MCP and authentication
- Inference latency, batching and GPU basics
- Evaluations, tracing, token cost and rate limits
- Prompt injection, data access and approvals

Build a small working system before exploring every framework.

Completion check: demonstrate one relevant task without copying commands blindly, then explain the expected result, one failure case and how you would investigate it.

AI TOOLS SERVICES

Advanced Track. Choose a small stack and understand each layer.

- Local models: Ollama or LM Studio
- Python APIs: FastAPI; agents: Agno
- MCP tools: FastMCP; retrieval: pgvector
- Serving: explore vLLM when GPU needs arise
- Cloud: Amazon Bedrock and AgentCore
- Azure: Microsoft Foundry; GCP: Vertex AI

Tools are examples, not mandatory prerequisites. Start locally if practical.

Completion check: demonstrate one relevant task without copying commands blindly, then explain the expected result, one failure case and how you would investigate it.

Capstone project plan

Build

Create a Python API, PostgreSQL-backed data flow and finite queue worker. Keep interfaces and migrations compatible.

Deliver

Create a small container image, provision a lab environment through IaC and build a pipeline that tests, scans and publishes an immutable artifact.

Operate

Add probes, workload identity, managed secrets, metrics, logs and traces. Define one useful alert and exercise a controlled failure.

Recover

Test a rollback and an isolated database restore. State what data is protected, what can be lost and how long recovery takes.

Explain

Publish a sanitized architecture, setup instructions, cost assumptions and a short incident report. Keep teardown instructions explicit.

Avoid common learning traps

- Do not learn three cloud consoles before understanding one network and identity model.
- Do not treat Kubernetes as mandatory for every application.
- Do not give AI agents broad production permissions to make a demo easier.
- Do not equate a green deployment or completed backup with validated service recovery.

Official learning starting points

Kubernetes concepts | AWS learning | Microsoft Learn | Google Cloud learning | OpenTelemetry | Model Context Protocol | Agno | Ollama | FastMCP |